

SELECT * FROM USER_DETAILS;

ID	ADDRESS_PIN_CODE	ADDRESS_STREET	NAME	PHONE
----	------------------	----------------	------	-------

(no rows, 2 ms)

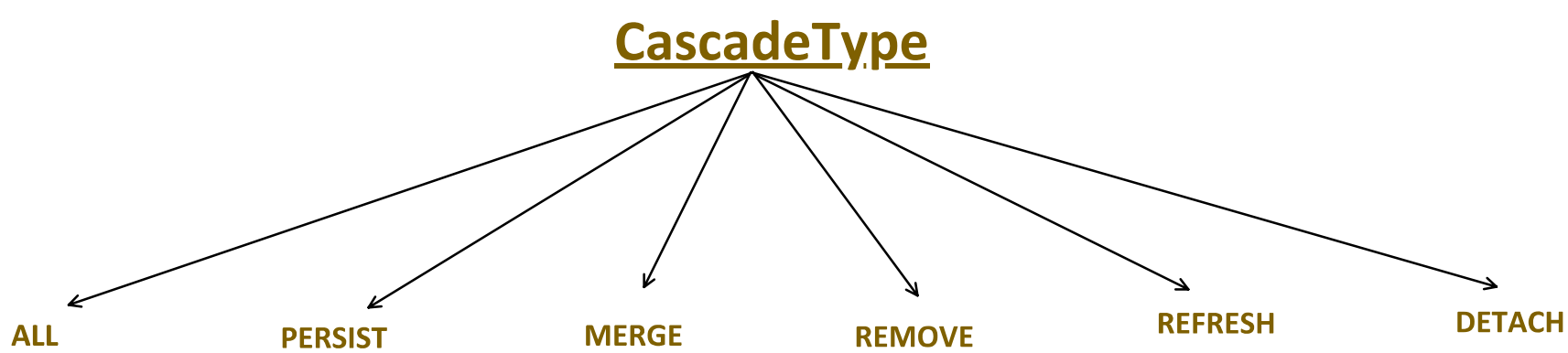
SELECT * FROM USER_ADDRESS;

CITY	COUNTRY	PIN_CODE	STATE	STREET
------	---------	----------	-------	--------

(no rows, 1 ms)

SELECT * FROM INFORMATION_SCHEMA.INDEX_COLUMNS:								
INDEX_CATALOG	INDEX_SCHEMA	INDEX_NAME	TABLE_CATALOG	TABLE_SCHEMA	TABLE_NAME	COLUMN_NAME	ORDINAL_POSITION	ORDERING_SPECIFICATION
USERDB	PUBLIC	PRIMARY_KEY_3	USERDB	PUBLIC	USER_DETAILS	ID	1	ASC
USERDB	PUBLIC	CONSTRAINT_INDEX_3	USERDB	PUBLIC	USER_DETAILS	ADDRESS_PIN_CODE	1	ASC
USERDB	PUBLIC	CONSTRAINT_INDEX_3	USERDB	PUBLIC	USER_DETAILS	ADDRESS_STREET	2	ASC
USERDB	PUBLIC	PRIMARY_KEY_9	USERDB	PUBLIC	USER_ADDRESS	PIN_CODE	1	ASC
USERDB	PUBLIC	PRIMARY_KEY_9	USERDB	PUBLIC	USER_ADDRESS	STREET	2	ASC

Now before we proceed with further Mappings, there is one more important thing i.e.



- Without CascadeType, any operation on Parent do not affect Child entity.
- Managing Child entities explicitly can be error-prone.

CascadeType.PERSIST

- Persisting/Inserting the User entity automatically persists its associated UserAddress entity data.

```
@Table(name = "user_details")
@Entity
public class UserDetails {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String phone;

    @OneToOne(cascade = CascadeType.PERSIST)
    @JoinColumn(name = "address_id", referencedColumnName = "id")
    private UserAddress userAddress;

    // Constructors
    public UserDetails() {
    }

    //getters and setters
}
```

```
@Entity
@Table(name = "user_address")
public class UserAddress {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String street;
    private String city;
    private String state;
    private String country;
    private String pinCode;

    //getters and setters
}
```

```
@RestController
@RequestMapping(value = "/api/")
public class UserController {

    @Autowired
    UserDetailsService userDetailsService;

    @PostMapping(path = "/user")
    public UserDetails insertUser(@RequestBody UserDetails userDetails) {
        return userDetailsService.saveUser(userDetails);
    }
}
```

```
@Service
public class UserDetailsService {

    @Autowired
    UserDetailsRepository userDetailsRepository;

    public UserDetails saveUser(UserDetails user) {
        return userDetailsRepository.save(user);
    }
}
```

```
@Repository
public interface UserDetailsRepository extends
    JpaRepository<UserDetails, Long> {
}
```

localhost:8080/api/user

POSTlocalhost:8080/api/user

ParamsAuthorizationHeaders (8)BodyScriptsSettings

noneform-datax-www-form-urlencodedrawbinaryGraphQLJSON

```
1 {
2   "name": "JohnXYZ",
3   "phone": "1234567890",
4   "userAddress": {
5     "street": "123 Street",
6     "city": "Bangalore",
7     "state": "Karnataka",
8     "country": "India",
9     "pinCode": "10001"
10  }
11 }
```

BodyCookiesHeaders (5)Test Results

PrettyRawPreviewVisualizeJSON

```
1 {
2   "id": 1,
3   "name": "JohnXYZ",
4   "phone": "1234567890",
5   "userAddress": {
6     "id": 1,
7     "street": "123 Street",
8     "city": "Bangalore",
9     "state": "Karnataka",
10    "country": "India",
11    "pinCode": "10001"
12  }
13 }
```

RunRun SelectedAuto completeClearSQL statement:

SELECT * FROM USER_DETAILS

SELECT * FROM USER_DETAILS;

ADDRESS_ID	ID	NAME	PHONE
------------	----	------	-------

11JohnXYZ1234567890
(1 row, 1 ms)

RunRun SelectedAuto completeClearSQL statement:

SELECT * FROM USER_ADDRESS

SELECT * FROM USER_ADDRESS;

ID	CITY	COUNTRY	PIN_CODE	STATE	STREET
----	------	---------	----------	-------	--------

1BangaloreIndia10001Karnataka123 Street
(1 row, 1 ms)

CascadeType.MERGE

- Updating the User entity automatically updates its associated UserAddress entity data.

Lets, first see, what happen if we use only **PERSIT** Cascade type

```
@RestController
@RequestMapping(value = "/api/")
public class UserController {

    @Autowired
    UserDetailsService userDetailsService;

    @PostMapping(path = "/user")
    public UserDetails insertUser(@RequestBody UserDetails userDetails) {
        return userDetailsService.saveUser(userDetails);
    }
}
```

```
@Service
public class UserDetailsService {

    @Autowired
    UserDetailsRepository userDetailsRepository;

    public UserDetails saveUser(UserDetails user) {
        return userDetailsRepository.save(user);
    }
}
```


- So, EntityManager has one method called "refresh".
- What it does is, for that entity directly read the value from DB instead of First Level Cache.

OneNote

```
entity manager of type <com.persistenceContextType, TRANSACTION> cascade
* @throws EntityNotFoundException if the entity no longer exists in the database
*/
public void refresh(Object entity);
```

So, When we use the CascadeType.REFRESH, we tell JPA to not only read Parent Entity from DB but also its associated Child Entities too.

So in this case, whenever on USERDETAILS entity, entityManager will internally invoke 'refresh() method', this Cascade type make sure that, USERADDRESS should also be read from DB not from First Level Cache.

```
@Table(name = "user_details")
@Entity
public class UserDetails {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String phone;

    @OneToOne(cascade = {CascadeType.REFRESH})
    @JoinColumn(name = "address_id", referencedColumnName = "id")
    private UserAddress userAddress;

    // Constructors
    public UserDetails() {
    }

    //getters and setters
}
```

CascadeType.DETACH

- Similarly like persist, remove, refresh method. EntityManager also has 'detach' method.
- Which purpose is to remove the given entity from the PERSISTENCE CONTEXT.
- Means JPA is not managing its lifecycle now.

```
/**
 * Remove the given entity from the persistence context, causing
 * a managed entity to become detached. Unflushed changes made
 * to the entity if any (including removal of the entity),
 * will not be synchronized to the database. Entities which
 * previously referenced the detached entity will continue to
 * reference it.
 * @param entity entity instance
 * @throws IllegalArgumentException if the instance is not an
 * entity
 * @since 2.0
 */
public void detach(Object entity);
```

So, When we use the CascadeType.DETACH, we tell JPA to not only detach Parent Entity from persistence context but also its associated Child Entities too.

So in this case, whenever on USERDETAILS entity, entityManager will internally invoke 'detach() method', this Cascade type make sure that, USERADDRESS should also be detached/removed from persistence context.

```
@Table(name = "user_details")
@Entity
public class UserDetails {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String phone;

    @OneToOne(cascade = {CascadeType.DETACH})
    @JoinColumn(name = "address_id", referencedColumnName = "id")
    private UserAddress userAddress;

    // Constructors
    public UserDetails() {
    }

    //getters and setters
}
```

CascadeType.ALL

- Means we want all the different Cascade types capabilities like PERSIST, MERGE, REMOVE, REFRESH & DETACH.

Okay, we saw INSERT, UPDATE, REMOVE operation, but what about GET?

Does child entities always get loaded with Parent Entity?

Eager Loading

Lazy Loading

- It means, associated entity is loaded immediately along with the parent entity.
 - Default for @OneToOne and @ManyToOne
- It means, associated entity is NOT loaded immediately.
 - Only loaded when explicitly accessed like when we call userDetail.getUserAddress().
 - Default for @OneToMany, @ManyToMany.

```
@Target({METHOD, FIELD})
@Retention(RUNTIME)
public @interface OneToOne {

    /**
     * (Optional) The entity class that is the target of the association.
     * Defaults to the type of the field or property that stores the association.
     */
    Class targetEntity() default void.class;

    /**
     * (Optional) The operations that must be cascaded to the target of the association.
     * By default no operations are cascaded.
     */
    CascadeType[] cascade() default {};

    /**
     * (Optional) Whether the association should be lazily
     * loaded or must be eagerly fetched. The EAGER
     * strategy is a requirement on the persistence provider runtime that
     * the associated entity must be eagerly fetched. The LAZY
     * strategy is a hint to the persistence provider runtime.
     */
    FetchType fetch() default FetchType.EAGER;
```

```
@Target({METHOD, FIELD})
@Retention(RUNTIME)
public @interface OneToMany {

    /**
     * (Optional) The entity class that is the target of the association. Optional only if the collection
     * property is defined using Java generics. Must be specified otherwise.
     * Defaults to the parameterized type of the collection when defined using generics.
     */
    Class targetEntity() default void.class;

    /**
     * (Optional) The operations that must be cascaded to the target of the association.
     * Defaults to no operations being cascaded.
     * When the target collection is a java.util.Map, the cascade element applies to the map value.
     */
    CascadeType[] cascade() default {};

    /**
     * (Optional) Whether the association should be lazily loaded or must be eagerly fetched. The EAGER
     * strategy is a requirement on the persistence provider runtime that the associated entities must be
     * eagerly fetched. The LAZY strategy is a hint to the persistence provider runtime.
     */
    FetchType fetch() default FetchType.LAZY;
```


(Optional) The entity class that is the target of the association. Optional only if the collection-valued relationship property is defined using Java generics. Must be specified otherwise.

(Optional) The parameterized type of the collection when defined using generics.

```
class targetEntity() default void class;
```

(Optional) The operations that must be cascaded to the target of the association.

When the target collection is a [Java.util.List](#), the `cascade` element applies to the map value.

Defaults to no operations being cascaded.

```
CascadeType[] cascade() default {};
```

(Optional) Whether the association should be lazily loaded or must be eagerly fetched. The EAGER strategy is a requirement on the association's provider, while the associated collection must be eagerly fetched. The LAZY strategy is a hint to the persistence provider, runtime.

```
FetchType lazy() default FetchType.LAZY;
```

- But here, there can be many child entities present for a parent entity, so returning all the rows of the child entity might impact performance, so by-default it uses LAZY technique.

We can also control this default behavior:

```
@Table(name = "user_details")
@Entity
public class UserDetails {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String phone;

    @OneToOne(cascade = CascadeType.ALL, fetch = FetchType.LAZY)
    @JoinColumn(name = "address_id", referencedColumnName = "id")
    private UserAddress userAddress;

    // Constructors
    public UserDetails() {
    }

    //getters and setters
}
```

Lets try testing this code:

2nd Get Operation: (Failure)

GET localhost:8080/api/users/

Params Authorization Headers (6) Body Scripts Settings

Query Params

Key	Value	Description
key	value	Description

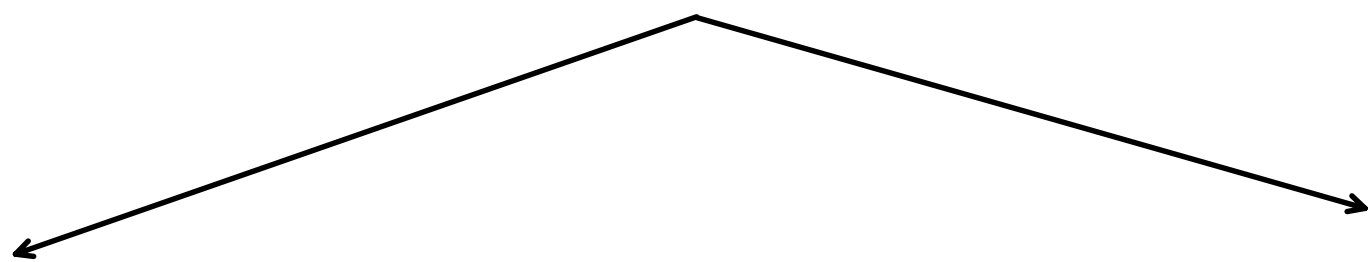
GET operation failed because of loading UserAddress LAZILY, Since during JSON creation at the time of response, UserAddress data is missing thus library like JAKSON don't know how to serialize it and thus it failed while constructing the response.

```
com.fasterxml.jackson.databind.exc.InvalidDefinitionException: Create breakpoint : No serializer found for class org.hibernate.proxy.pojo.bytebuddy.ByteBuddyInterceptor
    at com.fasterxml.jackson.databind.exc.InvalidDefinitionException.from(InvalidDefinitionException.java:77) ~[jackson-databind-2.17.2.jar:2.17.2]
    at com.fasterxml.jackson.databind.SerializerProvider.reportBadDefinition(SerializerProvider.java:1339) ~[jackson-databind-2.17.2.jar:2.17.2]
    at com.fasterxml.jackson.databind.DataBindContext.reportBadDefinition(DataBindContext.java:414) ~[jackson-databind-2.17.2.jar:2.17.2]
```

Hey, then how come INSERT operation, we don't see this serialization issue?

Its because during INSERT, we are inserting the data in DB and also its inserted into persistence Context (first level cache), so UserAddress data is already present in-memory. So when response is returned as part of same INSERT operation, it do not make any DB call, just fetched from the persistence context.

So, for GET operation, how to solve it:



- Use **@JsonIgnore**
- This will remove the UserAddress field totally for both Lazy and Eager loading.

```

@Table(name = "user_details")
@Entity
public class UserDetails {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String phone;

    @OneToMany(cascade = CascadeType.ALL, fetch = FetchType.LAZY)
    @JoinColumn(name = "address_id", referencedColumnName = "id")
    @JsonIgnore
    private UserAddress userAddress;

    // Constructors
    public UserDetails() {
    }

    //getters and setters
}

```

- Using **DTO (Data Transfer Object)**
- Much clean and recommended approach.
- Instead of sending Entity directly, first response will be mapped to our DTO object.

```
public class UserDetailsDTO {

    private Long id;
    private String name;
    private String phone;
    private String address;

    // Constructor to populate from UserDetails entity
    public UserDetailsDTO(UserDetails userDetails) {
        this.id = userDetails.getId();
        this.name = userDetails.getName();
        this.phone = userDetails.getPhone();
        System.out.println("going to query user address here now");
        this.address = userDetails.getUserAddress() != null ?
            userDetails.getUserAddress().getStreet() : null;
    }

    //getters and setters
}
```

```

@Table(name = "user_details")
@Entity
public class UserDetails {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String phone;

    @OneToOne(cascade = CascadeType.ALL, fetch = FetchType.LAZY)
    @JoinColumn(name = "address_id", referencedColumnName = "id")
    private UserAddress userAddress;

    // Constructors
    public UserDetails() {
    }

    public UserDetailsDTO toDTO() {
        return new UserDetailsDTO(this);
    }

    //getters and setters
}

```

GET

localhost:8080/api/user/1

Params

Authorization

Headers (6)

Body

Scripts

Settings

Query Params

Key	Value
Key	Value

A screenshot of the Chrome DevTools console. The top bar shows tabs for 'Body', 'Cookies', 'Headers (5)', 'Test Results', and a refresh icon. Below the tabs, there are buttons for 'Pretty', 'Raw', 'Preview', and 'Visualize'. To the right of these buttons is a dropdown menu currently set to 'JSON' and a red 'x' icon. The console area displays a JSON object:

```
{
  "id": 1,
  "name": "JohnXYZ",
  "phone": "1234567890"
}
```

 The 'id' property is highlighted with a mouse cursor.

```
@RestController
@RequestMapping(value = "/api/")
public class UserController {

    @Autowired
    UserDetailsService userDetailsService;

    @PostMapping(path = "/user")
    public UserDetails insertUser(@RequestBody UserDetails userDetails) {
        return userDetailsService.saveUser(userDetails);
    }

    @GetMapping("/user/{id}")
    public UserDetailsDTO fetchUser(@PathVariable Long id) {
        return userDetailsService.findByID(id).toDTO();
    }
}
```

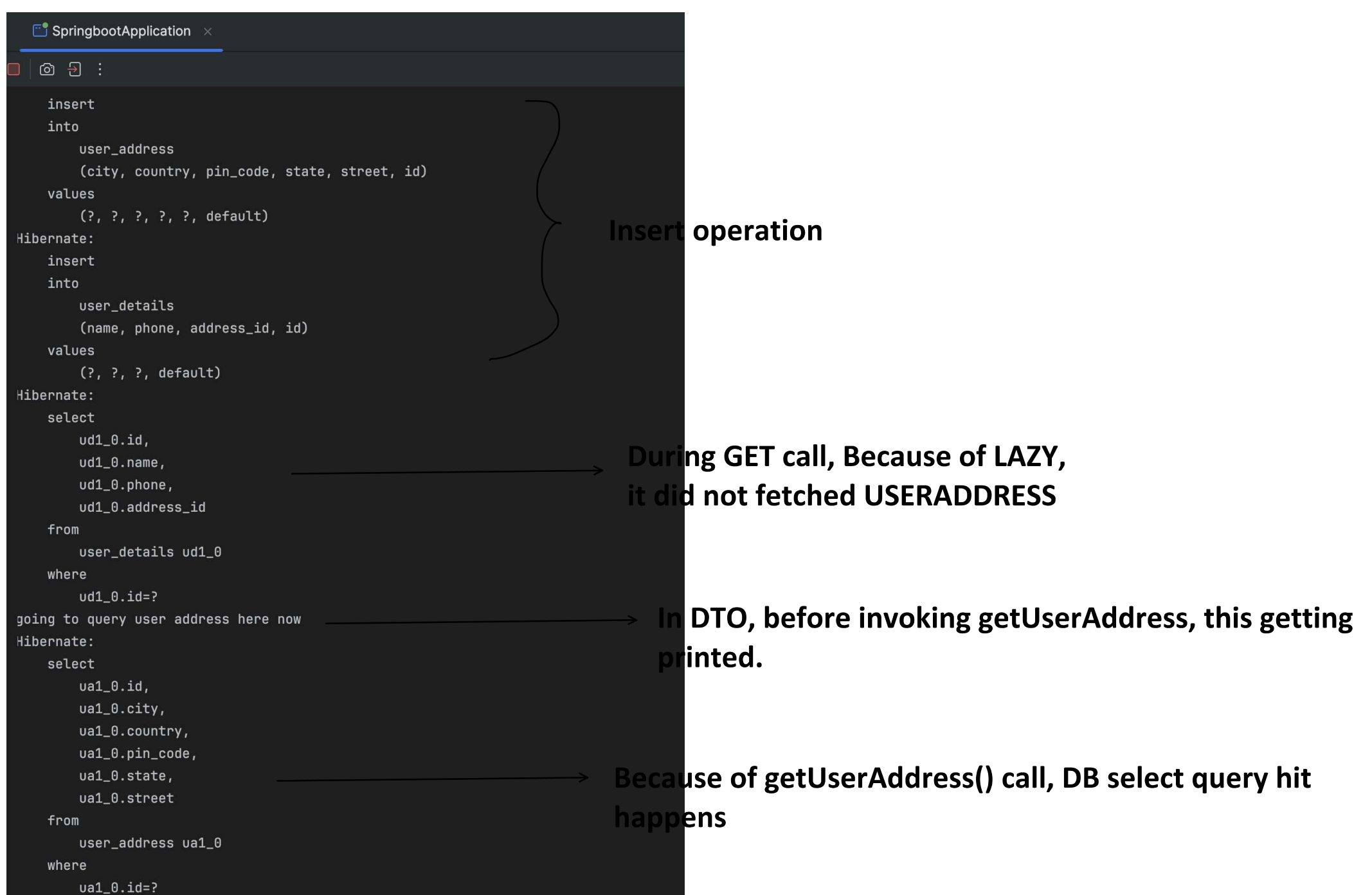
```
@Service
public class UserDetailsService {

    @Autowired
    UserDetailsRepository userDetailsRepository;

    public UserDetails saveUser(UserDetails user) {
        return userDetailsRepository.save(user);
    }

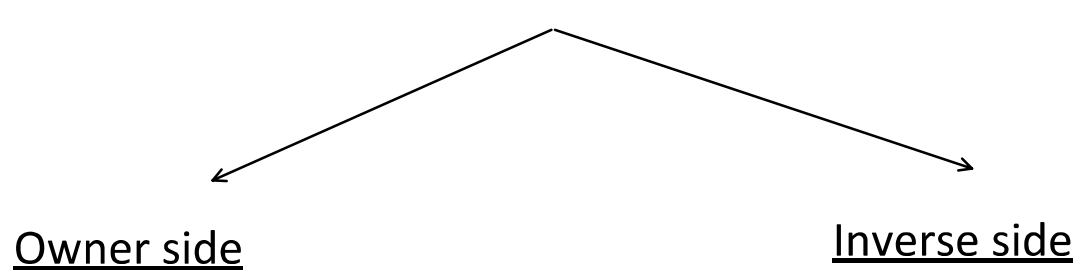
    public UserDetails findById(Long primaryKey) {
        return userDetailsRepository.findById(primaryKey).get();
    }
}
```

The screenshot shows the REST Client interface. At the top, the URL bar displays "localhost:8080/api/user/1". Below the URL bar, the "GET" method is selected. The "Query Params" section is empty. The "Body" tab is selected, and the response is displayed in JSON format: {"id": 1, "name": "JohnXYZ", "phone": "1234567890", "address": "123 Street"}. The response is highlighted in a light blue box.



@OneToOne bidirectional

- Both entities hold reference to each other, means:
 - UserDetails has a reference to UserAddress.
 - UserAddress also has a reference back to UserDetails (only in Object, not in DB table)



- Holds the Foreign Key relationship
- No Foreign key is created in table.

- Only holds **Object** reference of owing entity.

```
@Entity
@Table(name = "user_address")
public class UserAddress {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String street;
    private String city;
    private String state;
    private String country;
    private String pinCode;

    @ManyToOne(mappedBy = "userAddress", fetch = FetchType.EAGER)
    private UserDetails userDetails;

    //getters and setters
}
```

Table looks exactly same as OneToOne Unidirectional only.

~~address_id is a FK~~

But we now have capability to go backward from `UserAddress` to `UserDetails`.

Created a controller and service class to query UserAddress entity

```
@Service
public class UserAddressService {

    @Autowired
    UserAddressRepository userAddressRepository;

    public UserAddress findById(Long primaryKey) {
        return userAddressRepository.findById(primaryKey).get();
    }
}
```

Let's observe the behavior now,

1st: Insert Operation like before, using UserDetails controller API:

POST /api/users

Params Authorization Headers (8) **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL ☐ JSON

```
1 {
2   "name": "JohnXYZ",
3   "phone": "1234567890",
4   "userAddress": {
5     "street": "123 Street",
6     "city": "Bangalore",
7     "state": "Karnataka",
8     "country": "India",
9     "pinCode": "10001"
10  }
11 }
```

```
2024-12-31T16:45:04.749+05:30 INFO 39163 --- [nio-8080-exec-2] o.s.web.servlet.DispatcherServlet : Completed initialization in 1 ms
Hibernate:
insert
into
  user_address
(city, country, pin_code, state, street, id)
values
  (?, ?, ?, ?, ?, default)
Hibernate:
insert
into
  user_details
(name, phone, address_id, id)
values
  (?, ?, ?, default)
```

2nd: Get call of UserAddress Controller API:



The screenshot shows the Chrome DevTools Network tab. A GET request to `localhost:8080/api/user-address/1` is selected. The request headers are visible, and the response body is empty.


```

21     "name": "3ohoxYZ",
22     "phone": "1234567890",
23     "userAddress": {
24       "id": 1,
25       "street": "123 Street",
26       "city": "Bangalore",
27       "state": "Karnataka",
28       "country": "India",
29       "pinCode": "58001",
30       "userDetails": {
31         "id": 1,
32         "name": "3ohoxYZ",
33         "phone": "1234567890",
34         "userAddress": {
35           "id": 1,
36           "street": "123 Street",
37           "city": "Bangalore",
38           "state": "Karnataka",
39           "country": "India",
40           "pinCode": "58001",
41           "userDetails": {
42             "id": 1,
43             "name": "3ohoxYZ",
44             "phone": "1234567890",
45             "userAddress": {
46               "id": 1,
47               "street": "123 Street",

```

```
select
    ua1_0.id,
    ua1_0.city,
    ua1_0.country,
    ua1_0.pin_code,
    ua1_0.state,
    ua1_0.street,
    ud1_0.id,
    ud1_0.name,
    ud1_0.phone
from
    user_address ua1_0
left join
    user_details ud1_0
        on ua1_0.id=ud1_0.address_id
where
    ua1_0.id=?
```

Successfully executed JOIN query from UserAddress to UserDetail table

```
2024-12-31T16:47:23.303+05:30 WARN 39163 --- [nio-8080-exec-6] .w.s.m.s.DefaultHandlerExceptionResolver : Ignoring exception, response committed already: org.springframework.http.converter.HttpMessageNotReadableException:
2024-12-31T16:47:23.303+05:30 WARN 39163 --- [nio-8080-exec-6] .w.s.m.s.DefaultHandlerExceptionResolver : Resolved [org.springframework.http.converter.HttpMessageNotReadableException:]
```

But exception comes during Response building

Why because :

During response construction:

1. Jackson starts serializing UserAddress after UserAddress is serialized.
2. It encounters UserDetails within UserAddress and starts serializing it. After UserDetails is serialized.
3. Inside UserDetails , it encounters UserAddress again and serialization keeps going on in loop.

Infinite Recursion issue in bidirectional mapping can be solved via:

@JsonManagedReference @JsonBackReference
and

@JsonManagedReference:

- Should be Used only in Owing entity.
- Tells explicitly Jackson to go ahead and serialize the child entity.

@JsonBackReference:

- Should only be Used with Inverse/Child entity.
- Tells explicitly Jackson to not serialize the parent entity.

```

@Table(name = "User_details")
@Entity
public class UserDetails {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String phone;

    @OneToOne(cascade = CascadeType.ALL)
    @JoinColumn(name = "address_id", referencedColumnName = "id")
    @JsonManagedReference
    private UserAddress userAddress;

    // Constructors
    public UserDetails() {
    }

    //getters and setters
}

```

```
@Entity
@Table(name = "user_address")
public class UserAddress {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String street;
    private String city;
    private String state;
    private String country;
    private String pinCode;

    @OneToOne(mappedBy = "userAddress", fetch = FetchType.EAGER)
    @JsonBackReference
    private UserDetails userDetails;

    //getters and setters
}
```

GET API call on Parent Entity "UserDetail"

localhost:8080/api/user/1

GET localhost:8080/api/user/1

Params Authorization Headers (6) Body Scripts Settings

Query Params

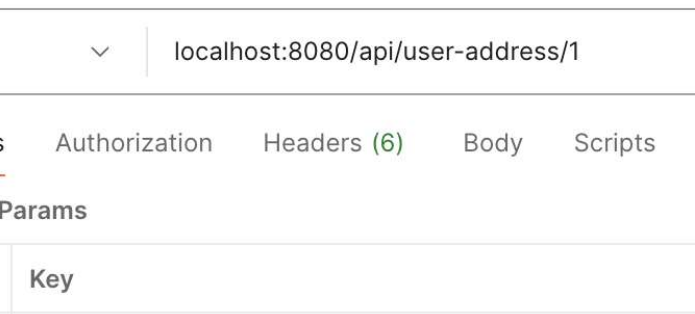
Key	Value
Key	

Body Cookies Headers (5) Test Results

Pretty Raw Preview Visualize JSON

```
1 {
2   "id": 1,
3   "name": "JohnXYZ",
4   "phone": "1234567890",
5   "userAddress": {
6     "id": 1,
7     "street": "123 Street",
8     "city": "Bangalore",
9     "state": "Karnataka",
10    "country": "India",
11    "pinCode": "19001"
12  }
13 }
```

GET API call on Child Entity "UserAddress"



The screenshot shows the Swagger UI for a REST API. The top bar displays the URL `localhost:8080/api/user-address/1` and the HTTP method `GET`. Below the URL bar, there are tabs for `Params`, `Authorization`, `Headers (6)`, `Body`, `Scripts`, and `Setting`. The `Query Params` section is expanded, showing a table with two columns: `Key` and `Value`. The `Body` tab is also visible. The `Pretty` tab is selected, displaying the JSON response in a formatted view. The response is a JSON object with the following fields: `id` (1), `street` ("123 Street"), `city` ("Bangalore"), `state` ("Karnataka"), `country` ("India"), and `pinCode` ("10001").

```

{
  "id": 1,
  "street": "123 Street",
  "city": "Bangalore",
  "state": "Karnataka",
  "country": "India",
  "pinCode": "10001"
}

```


Is there a way that, I can load the associated entity from both side, but still avoid infinite recursion:

@JsonIdentityInfo

- During serialization , Jackson gives the unique ID to the entity (based on property field).
- Though which Jackson can know, if that particular id entity is already serialized before, then it skip the serialization.

```
@Table(name = "user_details")
@Entity
@JsonIdentityInfo(
    generator = ObjectIdGenerators.PropertyGenerator.class,
    property = "id"
)
public class UserDetails {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String phone;

    @OneToOne(cascade = CascadeType.ALL)
    @JoinColumn(name = "address_id", referencedColumnName = "id")
    private UserAddress userAddress;

    // Constructors
    public UserDetails() {
    }

    //getters and setters
}
```

Any unique field from entity we can give, generally we give ID

```
@Entity
@Table(name = "user_address")
@JsonIdentityInfo(
    generator = ObjectIdGenerators.PropertyGenerator.class,
    property = "id"
)
public class UserAddress {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String street;
    private String city;
    private String state;
    private String country;
    private String pinCode;

    @OneToOne(mappedBy = "userAddress", fetch = FetchType.EAGER)
    private UserDetails userDetails;

    //getters and setters
}
```

GET API call on Parent Entity "UserDetail"

localhost:8080/api/user/1

GETlocalhost:8080/api/user/1

ParamsAuthorizationHeaders (6)BodyScriptsSetting

Query Params

Key
Key

BodyCookiesHeaders (5)Test Results

PrettyRawPreviewVisualizeJSON

```
1 {
2   "id": 1,
3   "name": "JohnXYZ",
4   "phone": "1234567890",
5   "userAddress": {
6     "id": 1,
7     "street": "123 Street",
8     "city": "Bangalore",
9     "state": "Karnataka",
10    "country": "India",
11    "pinCode": "10001",
12    "userDetails": 1
13  }
14 }
```

GET API call on Child Entity "UserAddress"

localhost:8080/api/user-address/1

GETlocalhost:8080/api/user-address/1

ParamsAuthorizationHeaders (6)BodyScriptsSetting

Query Params

Key
Key

BodyCookiesHeaders (5)Test Results

PrettyRawPreviewVisualizeJSON

```
1 {
2   "id": 1,
3   "street": "123 Street",
4   "city": "Bangalore",
5   "state": "Karnataka",
6   "country": "India",
7   "pinCode": "10001",
8   "userDetails": {
9     "id": 1,
10    "name": "JohnXYZ",
11    "phone": "1234567890",
12    "userAddress": 1
13  }
14 }
```